


**Herramientas Computacionales para Data Science**

---

# SVM y clasificadores clásicos

## Regresión logística y máquinas de vectores de soporte

Pedro Luiz Ramos

# Clasificación supervisada

Observamos pares $(x_i, y_i)$: predictores y una etiqueta conocida.

$$x_i = (x_{i1}, \ldots, x_{ip}), \qquad y_i \in \{0, 1\}.$$

- El entrenamiento usa ejemplos etiquetados para ajustar una regla.
- La regla se aplica a observaciones nuevas: $\widehat{y} = g(x)$.
- El objetivo es clasificar bien casos no utilizados en el ajuste.

**Una clase binaria no se transforma en una variable continua por codificarla como 0 y 1.**

# Salida del modelo y decisión

| Modelo | Salida | Regla de clasificación |
|---|---|---|
| Regresión logística | Probabilidad estimada $\widehat{p}(x)$ | Clase 1 si $\widehat{p}(x) \geq t$ |
| SVM | Puntaje de decisión $f(x)$ | Clase 1 si $f(x) \geq 0$ |

El umbral $t = 0.5$ es una referencia habitual, pero puede modificarse con un criterio definido previamente.

**El puntaje de una SVM no es, por sí solo, una probabilidad.**

# Caso de trabajo: Wisconsin

- 569 muestras: 357 benignas y 212 malignas.
- Variable respuesta: $Y = 1$ indica una muestra maligna.
- Usaremos cuatro de las 30 mediciones originales: radio, textura, suavidad y concavidad medias.
- Los datos y las particiones se entregan en un CSV común para R y Python.

**Es una base histórica para enseñanza; no representa a la población chilena ni constituye una herramienta clínica validada.**

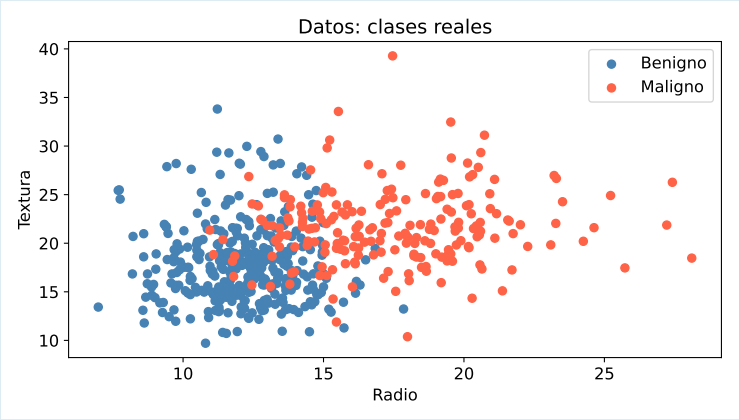
Fuente: Breast Cancer Wisconsin (Diagnostic), UCI; copia distribuida por scikit-learn.

# Entrenamiento y prueba

| Conjunto | Benigno (0) | Maligno (1) | Total |
|---|---|---|---|
| Entrenamiento | 250 | 148 | 398 |
| Prueba | 107 | 64 | 171 |
| Total | 357 | 212 | 569 |

- División estratificada: aproximadamente 70 % y 30 %.
- La columna `conjunto` ya indica qué filas corresponden a cada grupo.
- Usamos solo las cuatro mediciones como predictores.

# Dos predictores de la base




El gráfico del script muestra las 569 observaciones. Hay superposición entre clases. Los modelos usan cuatro predictores y se ajustan solo con entrenamiento.

# Referencia mínima: la clase mayoritaria

Una regla que siempre predice **benigno** acierta en 107 de los 171 casos de prueba:

$$\text{Exactitud} = \frac{107}{171} = 0.626.$$

Sin embargo, no detecta ninguna muestra maligna:

$$\text{Sensibilidad} = \frac{0}{64} = 0.$$

**Una exactitud aparentemente alta puede ocultar que una clase nunca se identifica.**

# Matriz de confusión

| | Predicha 0 | Predicha 1 |
|---|---|---|
| Real 0 | Verdadero negativo (VN) | Falso positivo (FP) |
| Real 1 | Falso negativo (FN) | Verdadero positivo (VP) |

- **FP**: un caso negativo se clasifica como positivo.
- **FN**: un caso positivo se clasifica como negativo.
- Siempre declaramos la clase positiva y el orden de filas y columnas.

# Métricas: ejemplo resuelto

Ejemplo ilustrativo: VN = 56, FP = 14, FN = 6 y VP = 24.

$$\begin{aligned}
\text{Exactitud} &= \frac{VP + VN}{n} = \frac{24 + 56}{100} = 0.80, \\
\text{Sensibilidad} &= \frac{VP}{VP + FN} = \frac{24}{30} = 0.80, \\
\text{Especificidad} &= \frac{VN}{VN + FP} = \frac{56}{70} = 0.80, \\
\text{Precisión} &= \frac{VP}{VP + FP} = \frac{24}{38} = 0.632.
\end{aligned}$$

Precisión responde: de los casos que declaramos positivos, ¿cuántos lo eran?

# F1, exactitud balanceada y AUC

$$F1 = \frac{2VP}{2VP + FP + FN} = \frac{48}{48 + 14 + 6} = 0.706,$$

$$\text{Exactitud balanceada} = \frac{\text{Sensibilidad} + \text{Especificidad}}{2}.$$

- F1 resume precisión y sensibilidad; no incorpora directamente VN.
- La exactitud balanceada da el mismo peso a ambas clases.
- AUC evalúa el ordenamiento de los puntajes a través de distintos umbrales.

**El ejemplo en R y Python compara exactitud, sensibilidad y especificidad.**

# Tres funciones distintas de los datos

| Etapa | Uso |
|---|---|
| Entrenamiento | Estimar coeficientes, escalas y vectores de soporte. |
| Validación | Comparar hiperparámetros, modelos y reglas de decisión. |
| Prueba | Evaluar el procedimiento que ya fue elegido. |

La actividad opcional realiza validación cruzada **dentro de los 398 casos de entrenamiento**. Los 171 casos de prueba permiten evaluar los modelos ya ajustados.

**Mirar repetidamente prueba para decidir convierte ese conjunto en validación.**

# Validación cruzada de cinco folds

| Iteración | Fold 1 | Fold 2 | Fold 3 | Fold 4 | Fold 5 |
|---|---|---|---|---|---|
| 1 | Validar | Ajustar | Ajustar | Ajustar | Ajustar |
| 2 | Ajustar | Validar | Ajustar | Ajustar | Ajustar |
| 3 | Ajustar | Ajustar | Validar | Ajustar | Ajustar |
| 4 | Ajustar | Ajustar | Ajustar | Validar | Ajustar |
| 5 | Ajustar | Ajustar | Ajustar | Ajustar | Validar |

Cada caso se usa una vez para validar. La actividad opcional compara el error medio de clasificación en R y la exactitud media en Python.

Dentro de cada búsqueda, todos los candidatos usan los mismos folds.

# Fuga de información

**Ejemplo incorrecto:** calcular medias y desviaciones con toda la base y después dividirla.

**Procedimiento correcto en cada fold:**

1. Estimar la transformación con los cuatro folds usados para ajustar.
2. Aplicar esa misma transformación al fold de validación.
3. Ajustar el clasificador y evaluar.

**La misma regla se aplica a imputación, selección de variables y sobremuestreo.**

# Estandarización de los predictores

Para la variable $j$:

$$z_{ij} = \frac{x_{ij} - \overline{x}_j^{\text{train}}}{s_j^{\text{train}}}.$$

- Evita que la unidad de medida domine las distancias de la SVM.
- Las medias y desviaciones se guardan para transformar datos nuevos.
- En R, `svm(..., scale = TRUE)` realiza la transformación.
- En Python, usamos `StandardScaler` antes del ajuste.

**Escalar ayuda a la geometría del problema; no elimina valores atípicos ni garantiza normalidad.**

# Regresión logística

Modelamos directamente la probabilidad de la clase positiva:

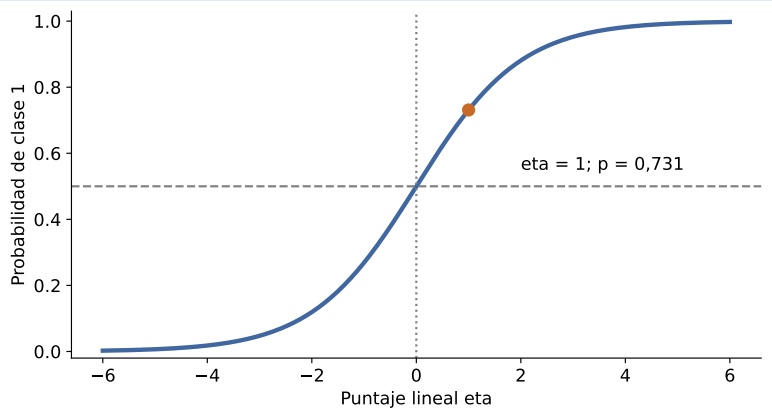
$$p(x) = P(Y = 1 \mid X = x), \qquad \log\left(\frac{p(x)}{1 - p(x)}\right) = \beta_0 + \beta^\top x.$$

Al despejar:

$$p(x) = \frac{1}{1 + \exp[-(\beta_0 + \beta^\top x)]}.$$

**El predictor es lineal en los coeficientes; la probabilidad queda entre 0 y 1.**

# La función logística




$\eta = 0$ corresponde a $p = 0.5$. Un aumento constante de $\eta$ no produce un aumento constante de la probabilidad.

# Predicción logística: paso a paso

Ejemplo didáctico, con coeficientes hipotéticos y variables estandarizadas:

$$\eta = -1 + 0.8z_1 + 0.4z_2, \qquad z_1 = 1.5, \quad z_2 = 2.$$

1. Puntaje: $\eta = -1 + 0.8(1.5) + 0.4(2) = 1$.
2. Probabilidad: $\widehat{p} = 1/(1 + e^{-1}) = 0.731$.
3. Con $t = 0.5$: $0.731 \geq 0.5 \Rightarrow \widehat{y} = 1$.
4. Con $t = 0.8$: $0.731 < 0.8 \Rightarrow \widehat{y} = 0$.

**La probabilidad no cambia al modificar el umbral; cambia la decisión.**

# Interpretación de los coeficientes

Los *odds* son $p/(1-p)$. Al aumentar $x_j$ en una unidad, manteniendo los demás predictores constantes:

$$\frac{\text{odds}_{\text{nuevo}}}{\text{odds}_{\text{anterior}}} = e^{\beta_j}.$$

Si $\beta_j = 0.8$, entonces $e^{0.8} = 2.23$: los odds se multiplican por 2.23.

- Esto no significa que la probabilidad se multiplique por 2.23.
- Con variables estandarizadas, una unidad es una desviación estándar del entrenamiento.
- La interpretación es una asociación condicional, no un efecto causal.

# Estimación y regularización

Los coeficientes se estiman minimizando la pérdida logística:

$$-\sum_{i=1}^{n} \left[ y_i \log(p_i) + (1 - y_i) \log(1 - p_i) \right].$$

- Una predicción muy segura y equivocada recibe una penalización grande.
- La regularización agrega una penalización a los coeficientes.
- Para comparar con `glm`, aquí usamos logística sin penalización en ambos idiomas.

`LogisticRegression()` aplica regularización por defecto: esa opción no reproduce automáticamente `glm(..., family=binomial())`.

# Frontera de decisión logística

Con el umbral $t = 0.5$:

$$\widehat{p}(x) \geq 0.5 \quad \Longleftrightarrow \quad \widehat{\beta}_0 + \widehat{\beta}^\top x \geq 0.$$

- Con dos predictores originales, la frontera es una recta.
- Con más predictores, es un hiperplano.
- Interacciones o transformaciones pueden generar fronteras no lineales en las variables originales.

**El ejemplo central usa efectos lineales para que la comparación sea fácil de seguir.**

# Datos en R

```
library(e1071)
setwd("D:/codigos")

datos <- read.csv("wisconsin_clase.csv")
print(head(datos))
print(table(datos$maligno))
```

- `library(e1071)` carga las funciones para SVM.
- `read.csv` lee la base incluida en la carpeta.
- `head` muestra las primeras filas y `table` cuenta las clases.

**La base tiene 357 casos benignos y 212 malignos.**

# Entrenamiento y prueba en R

```
entrenamiento <- subset(datos, conjunto == "entrenamiento")
prueba <- subset(datos, conjunto == "prueba")

variables <- c("radio", "textura", "suavidad", "concavidad")
x_entrenamiento <- entrenamiento[, variables]
x_prueba <- prueba[, variables]
y_entrenamiento <- factor(entrenamiento$maligno, levels = c(0, 1))
y_prueba <- factor(prueba$maligno, levels = c(0, 1))
```

- `x`: las cuatro variables que usamos para predecir.
- `y`: la etiqueta que queremos predecir.
- `factor` indica que 0 y 1 son clases.

# Datos en Python

```
import os
import pandas as pd

os.chdir(r"D:\codigos")

# %% 2. Leer y mirar los datos
datos = pd.read_csv("wisconsin_clase.csv")
print(datos.head())
print(datos["maligno"].value_counts())
```

- `pd.read_csv` crea una tabla de datos.
- `head()` muestra las primeras filas.
- `value_counts()` cuenta las observaciones de cada clase.

# Entrenamiento y prueba en Python

```
entrenamiento = datos[datos["conjunto"] == "entrenamiento"]
prueba = datos[datos["conjunto"] == "prueba"]

variables = ["radio", "textura", "suavidad", "concavidad"]
x_entrenamiento = entrenamiento[variables]
x_prueba = prueba[variables]
y_entrenamiento = entrenamiento["maligno"]
y_prueba = prueba["maligno"]
```

**Ambos lenguajes usan exactamente las mismas filas de entrenamiento y prueba.**

# Estandarización en Python

```
from sklearn.preprocessing import StandardScaler

escala = StandardScaler()
x_entrenamiento_escalado = escala.fit_transform(x_entrenamiento)
x_prueba_escalado = escala.transform(x_prueba)
```

- `fit_transform`: estima las medias y escalas con entrenamiento y transforma esos datos.
- `transform`: aplica las mismas medias y escalas a prueba.

**Para predecir un caso nuevo conservamos el objeto `escala`.**

# Regresión logística en R

```
logistica <- glm(maligno ~ radio + textura + suavidad + concavidad,
                 data = entrenamiento, family = binomial())
print(summary(logistica))

probabilidad <- predict(logistica, newdata = prueba, type = "response")
print(head(probabilidad))
```

- A la izquierda de ~ va la respuesta. A la derecha, los predictores.
- `family = binomial()` especifica la regresión logística.
- `type = "response"` entrega la probabilidad de maligno.

# Regresión logística en Python

```
from sklearn.linear_model import LogisticRegression

logistica = LogisticRegression(penalty=None, max_iter=2000)
logistica.fit(x_entrenamiento_escalado, y_entrenamiento)

probabilidad = logistica.predict_proba(x_prueba_escalado)[:, 1]
print(probabilidad[:6])
```

- `fit` estima los coeficientes con entrenamiento.
- `penalty=None` ajusta el modelo sin penalización.
- `[:, 1]` selecciona la probabilidad de la clase 1.

# Lectura del ajuste logístico

| Término | Coeficiente de `glm` en R |
|---|---|
| Intercepto | $-42.2511$ |
| Radio | 1.3399 |
| Textura | 0.4421 |
| Suavidad | 124.9794 |
| Concavidad | 21.4077 |

Una unidad adicional de textura multiplica los odds por $e^{0.4421} = 1.56$, manteniendo los otros predictores constantes.

R usa las unidades originales. Python estandariza las variables, por lo que sus coeficientes tienen otra escala. Las probabilidades estimadas son equivalentes salvo diferencias numéricas.

# De probabilidad a clase en R

```
clase_logistica <- factor(ifelse(probabilidad >= 0.5, 1, 0), levels = c(0, 1))
matriz_logistica <- table(Real = y_prueba, Prediccion = clase_logistica)
print(matriz_logistica)
```

$$\widehat{p} \geq 0.5 \Rightarrow \widehat{y} = 1, \qquad \widehat{p} < 0.5 \Rightarrow \widehat{y} = 0.$$

- `ifelse` aplica la regla a cada probabilidad.
- `table` cruza la clase real con la clase predicha.

**El ejemplo mantiene el umbral 0.5, fijado antes de evaluar prueba.**

# De probabilidad a clase en Python

```
from sklearn.metrics import confusion_matrix

clase_logistica = (probabilidad >= 0.5).astype(int)
matriz_logistica = confusion_matrix(y_prueba, clase_logistica, labels=[0, 1])
print("Logistica: filas = real; columnas = prediccion; orden = 0, 1")
print(matriz_logistica)
```

- `probabilidad >= 0.5` produce valores verdadero/falso.
- `astype(int)` los convierte en 1 y 0.
- Filas: clase real. Columnas: predicción. Orden: 0, 1.

# Máquinas de vectores de soporte

SVM busca una frontera que equilibre la separación entre clases y las violaciones del margen.

- **SVM lineal**: frontera lineal en el espacio de predictores.
- **SVM con kernel radial**: frontera flexible en el espacio original.
- La solución depende de los vectores de soporte y de los hiperparámetros.

**Usaremos etiquetas $-1$ y $+1$ para explicar la geometría; en los scripts mantenemos 0 y 1.**

# Hiperplano y función de decisión

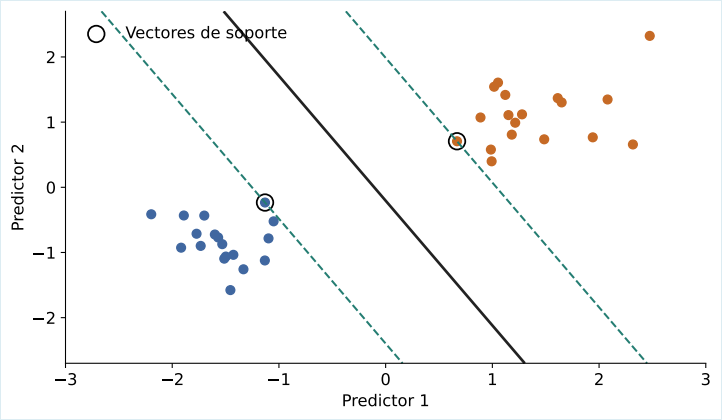
$$f(x) = w^\top x + b, \qquad \text{frontera: } f(x) = 0.$$

- $w$ determina la orientación del hiperplano.
- $b$ desplaza la frontera.
- El signo de $f(x)$ determina la clase predicha.

$$\widehat{y} = \begin{cases} +1, & f(x) \geq 0, \\ -1, & f(x) < 0. \end{cases}$$

**La distancia geométrica a la frontera es $|f(x)|/\|w\|$.**

# Margen y vectores de soporte




Datos simulados separables. Línea continua: $f(x) = 0$; discontinuas: $f(x) = \pm 1$. Los círculos grandes identifican los vectores de soporte del ajuste.

# Margen máximo: caso separable

Fijando la escala de $w$ y $b$ mediante $y_i f(x_i) \geq 1$:

$$\min_{w,b} \frac{1}{2}\|w\|^2 \quad \text{sujeto a} \quad y_i(w^\top x_i + b) \geq 1.$$

- Distancia de la frontera a cada borde: $1/\|w\|$.
- Ancho total del margen: $2/\|w\|$.
- Minimizar $\|w\|^2$ maximiza ese ancho.

**Un margen amplio no garantiza, por sí solo, buen rendimiento en datos nuevos.**

# Hiperplano: cuenta resuelta

Supongamos $w = (1, -1)$ y $b = -0.5$, en la escala usada para ajustar.

$$f(x) = x_1 - x_2 - 0.5, \qquad x = (2, 1).$$

1. Puntaje: $f(x) = 2 - 1 - 0.5 = 0.5$.
2. Clase: como $f(x) > 0$, predecimos $+1$.
3. Norma: $\|w\| = \sqrt{1^2 + (-1)^2} = \sqrt{2}$.
4. Distancia a la frontera: $0.5/\sqrt{2} = 0.354$.

Si su etiqueta real es $+1$, el caso queda bien clasificado, pero dentro del margen porque $0 < yf(x) < 1$.

# Margen suave y variables de holgura

$$\min_{w,b,\xi} \frac{1}{2}\|w\|^2 + C\sum_i \xi_i, \quad y_i f(x_i) \geq 1 - \xi_i, \quad \xi_i \geq 0.$$

En el óptimo, $\xi_i = \max\{0, 1 - y_i f(x_i)\}$.

| $y_i f(x_i)$ | $\xi_i$ | Lectura |
|---|---|---|
| 1.4 | 0 | Correcto y fuera del margen |
| 0.5 | 0.5 | Correcto, pero dentro del margen |
| $-0.3$ | 1.3 | Incorrectamente clasificado |

La holgura penaliza violaciones del margen, no solamente errores de clasificación.

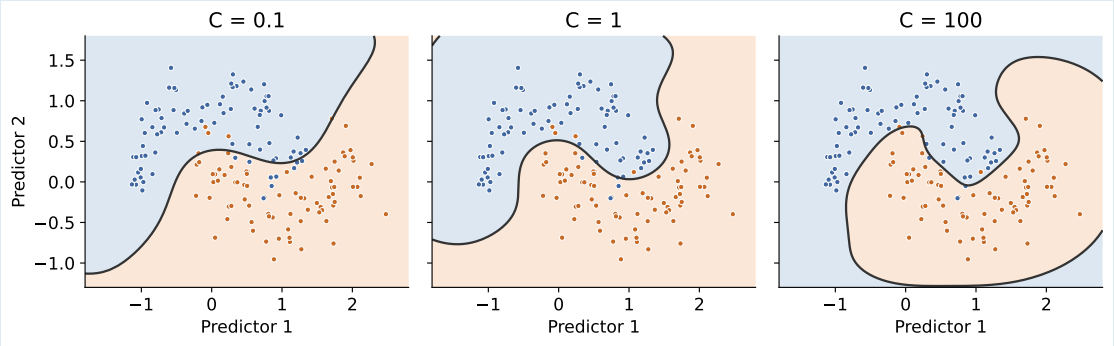
# El parámetro C

- $C$ **pequeño**: da menos peso a las violaciones; regularización más fuerte.
- $C$ **grande**: penaliza más las violaciones del entrenamiento.
- El efecto se interpreta manteniendo fijos el kernel, la escala y los datos.

Ejemplo: si $\sum_i \xi_i = 4$, la contribución de las holguras es $0.1(4) = 0.4$ con $C = 0.1$, y $10(4) = 40$ con $C = 10$.

**La validación decide cuánto conviene penalizar; un C mayor no siempre mejora el modelo.**

# Cambio de C con kernel radial fijo




Datos simulados; $\gamma = 1$ en los tres ajustes. Observe cómo cambia la frontera cuando aumenta la penalización de las violaciones.

# Funciones kernel

Un kernel calcula un producto interno en un espacio de características:

$$K(x, z) = \langle \phi(x), \phi(z) \rangle.$$

| | |
|---|---|
| Lineal | $K(x, z) = x^\top z$ |
| Polinomial | $K(x, z) = (\gamma x^\top z + r)^d$ |
| Radial (RBF) | $K(x, z) = \exp(-\gamma \|x - z\|^2)$ |

Con RBF, la frontera puede ser curva en el espacio original. No necesitamos construir explícitamente $\phi(x)$.

# Kernel radial: interpretación de gamma

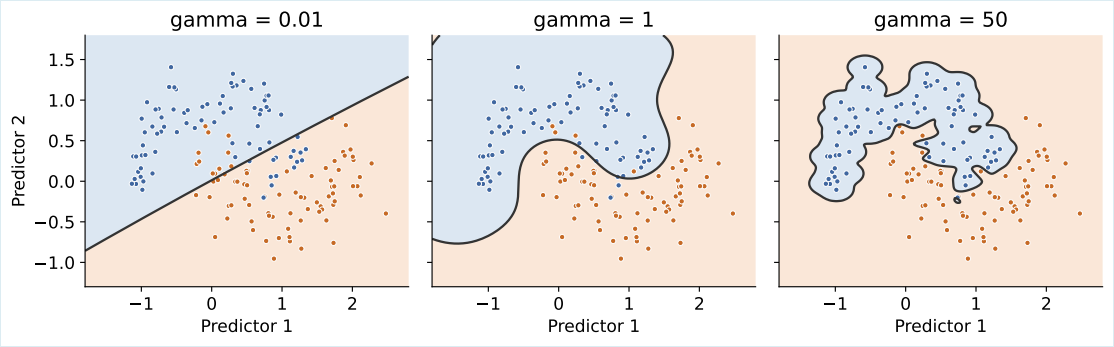
Para dos puntos separados por distancia 2 en el espacio estandarizado:

$$\|x - z\|^2 = 4, \qquad K(x, z) = e^{-4\gamma}.$$

$$\gamma = 0.1: \quad K = e^{-0.4} = 0.6703,$$
$$\gamma = 1: \quad K = e^{-4} = 0.0183.$$

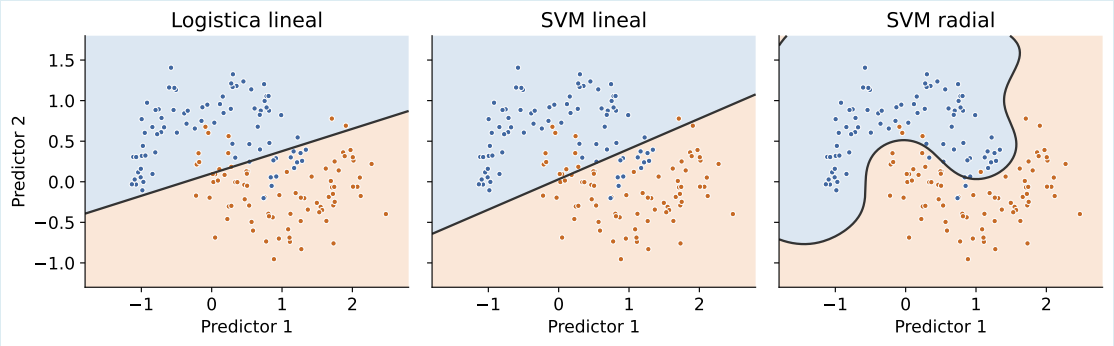
- $\gamma$ pequeño: la similitud decae más lentamente con la distancia.
- $\gamma$ grande: influencia más local; puede producir fronteras muy irregulares.

# Cambio de gamma con C fijo




Datos simulados; $C = 1$. Una frontera muy detallada puede ajustarse a particularidades del entrenamiento que no se repiten en datos nuevos.

# Comparación de fronteras




Mismos datos simulados. La logística con efectos lineales y la SVM lineal producen rectas; el kernel radial permite una frontera curva. Aquí se ilustra geometría, no rendimiento en prueba.

# Configuraciones del ejemplo

| Modelo | Configuración | Escalamiento |
|---|---|---|
| Logística | Sin penalización, $t = 0.5$ | En Python |
| SVM lineal | $C = 0.1$ | En ambos lenguajes |
| SVM radial | $C = 10$, $\gamma = 0.1$ | En ambos lenguajes |

Estos valores quedan fijados en el ejemplo principal. No los modificamos después de revisar los resultados de prueba.

**La actividad opcional muestra cómo comparar valores de C y gamma usando solo entrenamiento.**

# SVM lineal en R

```
svm_lineal <- svm(x = x_entrenamiento, y = y_entrenamiento,
                  kernel = "linear", cost = 0.1, scale = TRUE)
clase_lineal <- predict(svm_lineal, newdata = x_prueba)
matriz_lineal <- table(Real = y_prueba, Prediccion = clase_lineal)
print(matriz_lineal)
```

- `svm` recibe los predictores y las clases de entrenamiento.
- `kernel = "linear"` usa una frontera lineal.
- `scale = TRUE` estima y guarda la escala del entrenamiento.

`predict` **entrega directamente la clase 0 o 1.**

# SVM lineal en Python

```
from sklearn.svm import SVC

svm_lineal = SVC(kernel="linear", C=0.1)
svm_lineal.fit(x_entrenamiento_escalado, y_entrenamiento)
clase_lineal = svm_lineal.predict(x_prueba_escalado)
matriz_lineal = confusion_matrix(y_prueba, clase_lineal, labels=[0, 1])
print("SVM lineal: filas = real; columnas = prediccion; orden = 0, 1")
print(matriz_lineal)
```

**`C` en Python corresponde a `cost` en R.**

# SVM radial en R

```
svm_radial <- svm(x = x_entrenamiento, y = y_entrenamiento,
                  kernel = "radial", cost = 10, gamma = 0.1, scale = TRUE)
clase_radial <- predict(svm_radial, newdata = x_prueba)
matriz_radial <- table(Real = y_prueba, Prediccion = clase_radial)
print(matriz_radial)
```

- Cambiamos a `kernel = "radial"`.
- `cost = 10` fija la penalización de las violaciones del margen.
- `gamma = 0.1` fija el alcance de la similitud entre puntos.

# SVM radial en Python

```
svm_radial = SVC(kernel="rbf", C=10, gamma=0.1)
svm_radial.fit(x_entrenamiento_escalado, y_entrenamiento)
clase_radial = svm_radial.predict(x_prueba_escalado)
matriz_radial = confusion_matrix(y_prueba, clase_radial, labels=[0, 1])
print("SVM radial: filas = real; columnas = prediccion; orden = 0, 1")
print(matriz_radial)
```

- Python llama `"rbf"` al kernel radial.
- `fit` recibe entrenamiento ya estandarizado.
- `predict` aplica la regla a prueba en esa misma escala.

# Métricas de la SVM radial en R

```
mean(clase_radial == y_prueba)

matriz_radial[2, 2] / sum(matriz_radial[2, ])

matriz_radial[1, 1] / sum(matriz_radial[1, ])
```

- Primera cuenta: exactitud, proporción de aciertos.
- Segunda cuenta: sensibilidad, VP dividido por los malignos reales.
- Tercera cuenta: especificidad, VN dividido por los benignos reales.

Son las mismas cuentas que aparecen en la tabla `comparacion` del script.

# Métricas de la SVM radial en Python

```
from sklearn.metrics import accuracy_score, recall_score

accuracy_score(y_prueba, clase_radial)

recall_score(y_prueba, clase_radial, pos_label=1)

recall_score(y_prueba, clase_radial, pos_label=0)
```

- `accuracy_score` calcula la exactitud.
- `recall_score` con `pos_label=1` calcula sensibilidad.
- Con `pos_label=0`, calcula especificidad.

# Tabla de comparación en R

```
comparacion <- data.frame(
  Modelo = c("Logistica", "SVM lineal", "SVM radial"),
  Exactitud = c(mean(clase_logistica == y_prueba),
                mean(clase_lineal == y_prueba),
                mean(clase_radial == y_prueba)),
  Sensibilidad = c(matriz_logistica[2, 2] / sum(matriz_logistica[2, ]),
                   matriz_lineal[2, 2] / sum(matriz_lineal[2, ]),
                   matriz_radial[2, 2] / sum(matriz_radial[2, ])),
  Especificidad = c(matriz_logistica[1, 1] / sum(matriz_logistica[1, ]),
                    matriz_lineal[1, 1] / sum(matriz_lineal[1, ]),
                    matriz_radial[1, 1] / sum(matriz_radial[1, ]))
)
print(comparacion, digits = 3, row.names = FALSE)
```

Una fila por modelo. Cada columna contiene una de las métricas que acabamos de calcular.

# Tabla de comparación en Python

```
comparacion = pd.DataFrame({
    "Modelo": ["Logistica", "SVM lineal", "SVM radial"],
    "Exactitud": [accuracy_score(y_prueba, clase_logistica),
                  accuracy_score(y_prueba, clase_lineal),
                  accuracy_score(y_prueba, clase_radial)],
    "Sensibilidad": [recall_score(y_prueba, clase_logistica, pos_label=1),
                     recall_score(y_prueba, clase_lineal, pos_label=1),
                     recall_score(y_prueba, clase_radial, pos_label=1)],
    "Especificidad": [recall_score(y_prueba, clase_logistica, pos_label=0),
                      recall_score(y_prueba, clase_lineal, pos_label=0),
                      recall_score(y_prueba, clase_radial, pos_label=0)]
})
print(comparacion.round(3).to_string(index=False))
```

`pd.DataFrame` organiza los resultados en una tabla.

# Resultados en los 171 casos de prueba

| Modelo | Exactitud | Sensibilidad | Especificidad |
|---|---|---|---|
| Logística | 0.9415 | 0.9062 | 0.9626 |
| SVM lineal | 0.9181 | 0.8438 | 0.9626 |
| SVM radial | 0.9708 | 0.9375 | 0.9907 |

Logística usa $t = 0.5$. En SVM, `predict` aplica la regla de clasificación del modelo.

**Los dos scripts obtienen las mismas matrices de confusión y estas mismas métricas.**

Comparación docente de una partición: no implica superioridad universal ni validación clínica.

# SVM radial: cuenta final resuelta

| | Pred. 0 | Pred. 1 |
|---|---|---|
| Real 0 | 106 | 1 |
| Real 1 | 4 | 60 |

Filas: realidad.
Columnas: predicción.

$$\text{Exactitud} = \frac{106 + 60}{171} = 0.9708,$$

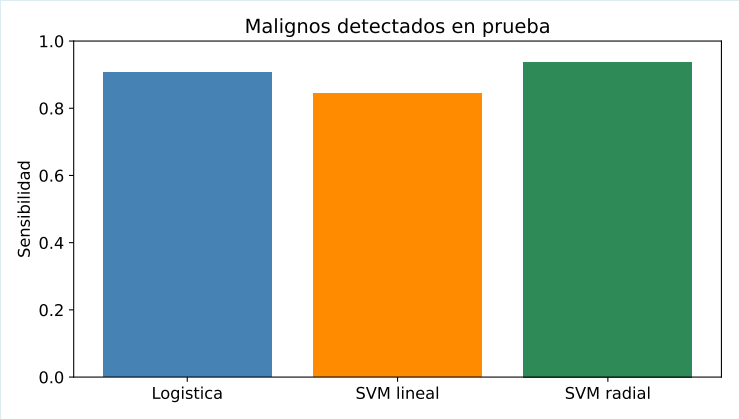
$$\text{Sensibilidad} = \frac{60}{60 + 4} = 0.9375,$$

$$\text{Especificidad} = \frac{106}{106 + 1} = 0.9907,$$

$$\text{Precisión} = \frac{60}{60 + 1} = 0.9836.$$

**En esta prueba hubo cuatro falsos negativos y un falso positivo.**

# Sensibilidad de los tres modelos




De los 64 casos malignos, la logística detecta 58, la SVM lineal 54 y la SVM radial 60. La tabla anterior también permite comparar especificidad.

# Clasificar observaciones nuevas en R

Ejemplo sintético, con las mismas definiciones y unidades de los predictores:

```
nuevo <- data.frame(radio = 14, textura = 20,
                    suavidad = 0.10, concavidad = 0.08)
print(predict(logistica, newdata = nuevo, type = "response"))
print(predict(svm_radial, newdata = nuevo))
```

- Logística: $\widehat{p} = 0.3919 < 0.5$, por lo tanto clase 0.
- SVM radial: clase 1.
- `predict` de SVM aplica la escala guardada durante el ajuste.

# Clasificar observaciones nuevas en Python

```
nuevo = pd.DataFrame({"radio": [14], "textura": [20],
                      "suavidad": [0.10], "concavidad": [0.08]})
nuevo_escalado = escala.transform(nuevo)
print(logistica.predict_proba(nuevo_escalado)[:, 1])
print(svm_radial.predict(nuevo_escalado))
```

- `escala.transform` usa las medias y escalas del entrenamiento.
- Logística: $\widehat{p} \simeq 0.3919$, clase 0. SVM radial: clase 1.

**Los modelos pueden discrepar porque sus fronteras son distintas.**

# Regresión logística o SVM

| Criterio | Logística | SVM lineal | SVM radial |
|---|---|---|---|
| Salida | Probabilidad | Puntaje | Puntaje |
| Frontera aquí | Lineal | Lineal | No lineal |
| Interpretación | Coeficientes y odds | Pesos y margen | Más difícil |
| Ajustes | Términos y penalización | C y escala | C, gamma y escala |
| Uso práctico | Interpretación | Separación lineal | Patrones curvos |

**La elección combina validación, interpretabilidad, costo de los errores y costo computacional.**

# Errores frecuentes al implementar

- No verificar cuál etiqueta representa la clase positiva.
- Escalar toda la base antes de hacer validación cruzada.
- Elegir C, gamma o el umbral mirando prueba.
- Pasar etiquetas 0/1 a una función de AUC en vez de puntajes.
- Interpretar `decision_function` como probabilidad.
- Comparar modelos con particiones distintas y atribuir toda diferencia al algoritmo.

# Decisiones para una aplicación nueva

1. Definir la etiqueta positiva y los errores relevantes.
2. Reservar prueba y construir folds apropiados para los datos.
3. Ajustar el preprocesamiento dentro de cada fold.
4. Comparar logística, SVM lineal y SVM radial.
5. Elegir configuración y umbral con validación.
6. Evaluar una vez en prueba y documentar las limitaciones.

**Con datos temporales o varias filas por persona, los folds deben respetar tiempo o grupos.**

# Referencias y material de apoyo

- Material base: Pedro Luiz Ramos, *Clase 06*, Diplomado de Métodos Estadísticos y Aplicaciones (2025); código `classe6.R`.
- Material del programa: Acosta, Castro y Moraga, *Clase 2*, secciones de validación cruzada y SVM. Identidad visual Data Science UC.
- scikit-learn: SVM; regresión logística; prevención de fuga de información.
- R: e1071::svm; R: stats::glm.
- UCI: Breast Cancer Wisconsin (Diagnostic).

`clasificadores.R`, `clasificadores.py` y `wisconsin_clase.csv` están juntos en:

`D:/codigos`

# Opcional: validación cruzada en R

```
set.seed(2026)
busqueda <- tune.svm(x = x_entrenamiento, y = y_entrenamiento,
                     kernel = "radial", scale = TRUE,
                     cost = c(0.1, 1, 10), gamma = c(0.01, 0.1, 1),
                     tunecontrol = tune.control(cross = 5))
print(busqueda$best.parameters)
```

- Nueve combinaciones y cinco folds: 45 ajustes.
- `tune.svm` compara el error medio de clasificación.
- El escalamiento se estima dentro de cada fold.

Este bloque está comentado al final del script y usa solo entrenamiento.

# Opcional: validación cruzada en Python

```
from sklearn.pipeline import make_pipeline
from sklearn.model_selection import GridSearchCV
modelo = make_pipeline(StandardScaler(), SVC(kernel="rbf"))
busqueda = GridSearchCV(modelo,
                        {"svc__C": [0.1, 1, 10], "svc__gamma": [0.01, 0.1, 1]},
                        cv=5, scoring="accuracy")
busqueda.fit(x_entrenamiento, y_entrenamiento)
print(busqueda.best_params_)
```

El pipeline estima la escala dentro de cada fold. Por eso recibe los predictores originales, sin escalar. `svc__C` y `svc__gamma` identifican los parámetros de SVC.

# Opcional: lectura de la búsqueda

| Lenguaje | C | Gamma | Criterio |
|---|---|---|---|
| R | 1 | 0.1 | Menor error medio |
| Python | 10 | 0.1 | Mayor exactitud media |

- Exactitud y error son complementarios: $\text{exactitud} = 1 - \text{error}$.
- Los scripts no construyen los mismos folds internos en ambos lenguajes.
- R y Python también usan convenciones levemente distintas para la desviación estándar.

**La partición interna puede cambiar el valor elegido. Esta actividad no modifica los modelos del ejemplo principal.**

# Apoyo: interpretación de la AUC

La AUC compara puntajes de casos positivos y negativos:

$$\mathrm{AUC} = P(S_1 > S_0) + \tfrac{1}{2}P(S_1 = S_0).$$

- Una AUC cercana a 1 indica un buen ordenamiento entre clases.
- No es la probabilidad de que cada clasificación sea correcta.
- No define el umbral operativo ni mide calibración de probabilidades.
- Dos modelos pueden tener la misma AUC y distinta sensibilidad en su corte habitual.

Es un complemento conceptual. Los scripts de esta clase no calculan AUC.

# Apoyo: más de dos clases

- La regresión logística puede extenderse al caso multinomial.
- Una SVM binaria puede combinarse mediante uno contra uno o uno contra el resto.
- `SVC` entrena internamente clasificadores uno contra uno.
- En `iris`, la respuesta tiene tres especies y debe conservarse como categórica.

**La evaluación debe indicar cómo se promedian las métricas entre clases: macro, micro o ponderado.**

Este material es de consulta; la demostración central mantiene dos clases.

# Apoyo: pérdida logística y hinge

Con $y \in \{-1, +1\}$ y puntaje $f(x)$:

$$\ell_{\log}(y, f) = \log(1 + e^{-yf}),$$
$$\ell_{\text{hinge}}(y, f) = \max(0, 1 - yf).$$

- Ambas penalizan puntajes de signo incorrecto.
- Hinge vale cero cuando el caso queda más allá del margen correcto.
- La logística conserva una contribución positiva para puntajes finitos.

**Las fronteras pueden parecer similares, pero los criterios de ajuste son distintos.**

# Apoyo: validación cruzada anidada

Cuando se necesita evaluar todo el proceso de búsqueda sin un test separado:

1. El ciclo externo reserva casos para evaluar.
2. El ciclo interno selecciona C, gamma y otras decisiones usando solo el entrenamiento externo.
3. Se reajusta la configuración elegida y se evalúa en el fold externo.

**La métrica usada para elegir el mejor candidato no es una evaluación independiente de esa elección.**

La actividad opcional usa cinco folds internos. La base mantiene un conjunto de prueba reservado.

# Apoyo: datos faltantes y categorías

- La base usada en la demostración no tiene valores faltantes en las cuatro variables seleccionadas.
- En otra base, la imputación debe ajustarse dentro de cada fold.
- Las variables categóricas requieren una codificación coherente entre entrenamiento y datos nuevos.
- No se debe sustituir automáticamente cada dato faltante por cero.

**En Python, `ColumnTransformer` y `Pipeline` permiten agrupar estas operaciones; en R pueden integrarse con recetas de preprocesamiento.**